Jacobian의 계산

교재의 내용을 따라, 다음과 같이 코드를 작성함.

$$ {}^0J = \begin{bmatrix} -l_1 s_1 - l_2 s_{12} & -l_2 s_{12} \\ l_1 c_1 + l_2 c_{12} & l_2 c_{12} \end{bmatrix} $$

```
function J = jacobian_2dof(theta1, theta2, L1, L2)
    J = [
        -L1*sin(theta1) - L2*sin(theta1 + theta2), -L2*sin(theta1 + theta2);
         L1*cos(theta1) + L2*cos(theta1 + theta2),  L2*cos(theta1 + theta2)
    ];
end
```

damped pseudo-inverse Jacobian의 계산

$\dot{\boldsymbol{x}} = \boldsymbol{J}\dot{\boldsymbol{q}}$(1). Singularities at $\theta_2 = 0$ or $\theta_2 = \pi$

$$ \boldsymbol{J}^+ = \boldsymbol{J}^{\boldsymbol{T}}(\boldsymbol{J}\boldsymbol{J}^{\boldsymbol{T}} + \lambda \boldsymbol{I})^{-1} $$

Where $\boldsymbol{J}^+$ is the pseudo inverse of $\boldsymbol{J}$
λ (damping coefficient) is chosen as follows:

$$ \lambda = \begin{cases} 0, \sigma \geq \sigma_d \\ \sqrt{\sigma(\sigma - \sigma_d)}, \frac{\sigma_d}{2} \leq \sigma \leq \sigma_d \\ \frac{\sigma_d}{2}, \sigma \leq \frac{\sigma_d}{2} \end{cases} $$

Where σ_d is the maximum singular region (chosen), σ can be choose as

$$ \sigma = \min(\|\theta_2\|, \|\theta_2 - \pi\|) $$

```
function J_damped_inv = damped_pseudoinverse(J, sigma, sigma_d)
    if sigma >= sigma_d
        lambda = 0;
    elseif sigma >= sigma_d/2
        lambda = sqrt(sigma * (sigma_d - sigma));
    else
        lambda = sigma_d/2;
    end

    % Pseudoinverse 계산
    I = eye(size(J,1));
    J_damped_inv = J' * inv(J*J' + (lambda)*I);
end
```

test_velControl.m 의 수정

$$\dot{q} = J^{+}\dot{x} + (I - J^{+}J)v_0$$

```
clear all

l1 = 1; l2 = 1;
I1 = 1; I2 = 1;
m1=1; m2 = 1;

% initialize random desired joint velocity
N = 10;
q_dotD = rand(2,N)/2;

theta10 = deg2rad(45); theta20 = deg2rad(45); % 초기값 변경
theta10_dot = 0; theta20_dot = 0; % initial joint velocities

% send to simulink model to get the end effector results

p = zeros(2,N); % end effector position
p_dot =zeros(2,N); % end effector vel
q = zeros(2,N); % joint angle
q_dot =zeros(2,N); % joint vel

% 각 영상 프레임을 그리기위한 plot설정
figure;
set(gcf,'Position',[100 100 700 520]);
axis([-2.5 2.5 -2.5 2.5]); % <-- 강제 axis 설정
axis equal
grid on
hold on
xlabel('X [m]');
ylabel('Y [m]');
title('2DoF Planar Robot Motion');

for i=1:N
    simIn = Simulink.SimulationInput("robotDynModelVelControl"); %create object

    % Jacobian 계산
    J = jacobian_2dof(theta10, theta20, l1, l2);

    condJ(i) = cond(J);
    disp(['Step ', num2str(i), ' - Condition Number: ', num2str(condJ(i))]);

    % sigma 계산
    sigma = compute_sigma(theta20);
    sigma_d = deg2rad(20);
```

```
J_damped_inv = damped_pseudoinverse(J, sigma, sigma_d);

qdot_des = q_dotD(:,i);

% Null space
k = 0.4; % gain 조절 가능
v0 = -k * q(:,i);

I = eye(size(J,2)); % n x n identity
nullspace_projector = (I - J_damped_inv * J);

qd_dot = J_damped_inv * qdot_des + nullspace_projector * v0;

%qd_dot = q_dotD(:,i);
simIn = simIn.setVariable('qd_dot', qd_dot);

out = sim(simIn); %run simulation, all results returned in "out"
% end effector position
p(:,i) = out.p.data(:,:,end);
p_dot(:,i) = out.p_dot.data(:,:,end);
q(:,i) = out.q.data(:,:,end);
q_dot(:,i) = out.q_dot.data(:,:,end);
% update initial joint angles and joinvelocities
theta10 = q(1,end);
theta20 = q(2,end);
theta10_dot = q_dot(1,end);
theta20_dot = q_dot(2,end);

% === 그림 그리기 ===
cla;

theta1 = q(1,i);
theta2 = q(2,i);

x1 = l1 * cos(theta1);
y1 = l1 * sin(theta1);
x2 = x1 + l2 * cos(theta1 + theta2);
y2 = y1 + l2 * sin(theta1 + theta2);
disp(['x1 = ', num2str(x1), ', y1 = ', num2str(y1)]);
disp(['x2 = ', num2str(x2), ', y2 = ', num2str(y2)]);

% 링크 그리기
plot([0, x1], [0, y1], 'b-', 'LineWidth', 1);
plot([x1, x2], [y1, y2], 'r-', 'LineWidth', 1);
plot(x2, y2, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k');

axis equal
xlim([-2.5 2.5]);
ylim([-2.5 2.5]);
grid on
title(['Step ', num2str(i)]);
drawnow;
```

```
    % === 그림 파일로 저장 ===
    filename = sprintf('step_%02d.png', i); % step_01.png, step_02.png, ...
    exportgraphics(gcf, filename, 'Resolution', 300);
end

close all
subplot(2,1,1)
plot(q_dot(1,:),'LineWidth',1.5)
hold on
plot(q_dotD(1,:))
legend('real', 'des','LineWidth',1.5)
title('joint 1')

subplot(2,1,2)
plot(q_dot(2,:),'LineWidth',1.5)
hold on
plot(q_dotD(2,:),'LineWidth',1.5)
legend('real', 'des')
title('joint 12')
```

해당 코드를 실행하여 결과를 확인하였으나, Damped 된 특성은 확인하였지만, Singularity 문제를 아직 해결 하지 못하였습니다.